SPEC VERIFICATION LAB

Khung thực hành Đặc tả → Kripke/CTL → Sinh mã → CWE/CVE

HƯỚNG DẪN TẠO DỮ LIỆU ĐẦU VÀO (INPUT) BẰNG FILE EXCEL (.xlsx) THEO TEMPLATE ĐẶC TẢ

Tài liệu hướng dẫn chi tiết từng mục — tất cả các mục là BẮT BUỘC

MỤC LỤC

MỤC LỤC 2
Chương 1 — Giới thiệu và nguyên tắc chung 4
1.1. Mục đích của tài liệu 4
1.2. Nguyên tắc áp dụng trong tài liệu này: TẤT CẢ các mục là bắt buộc 4
1.3. Kiến trúc tổng thể: từ Excel đến báo cáo CWE/CVE 4
Chương 2 — Cấu trúc tổng thể của Workbook (10 sheet) 6
Chương 3 — Hướng dẫn chi tiết từng sheet 7
3.1. Sheet “ThongTin” — Thông tin chung hệ thống 7
Cấu trúc cột 7
Kỹ thuật liên quan 7
Cách tính / quy tắc điền 7
Ví dụ minh họa (quanly_thuvien_muontra) 7
3.2. Sheet “YeuCauChucNang” — Yêu cầu chức năng 8
Cấu trúc cột 8
Kỹ thuật liên quan 8
Cách tính / quy tắc điền 8
Ví dụ minh họa (5 dòng đầu, quanly_thuvien_muontra) 8
3.3. Sheet “YeuCauPhiChucNang” — Yêu cầu phi chức năng 10
Cấu trúc cột 10
Kỹ thuật liên quan 10
Cách tính / quy tắc điền 10
Ví dụ minh họa (quanly_thuvien_muontra) 10
3.4. Sheet “RangBuoc” — Ràng buộc hệ thống 11
Cấu trúc cột 11
Kỹ thuật liên quan 11
Cách tính / quy tắc điền 11
Ví dụ minh họa (quanly_thuvien_muontra) — chú ý cột MoTa khớp KichThuocBuffer ở mục 3.8 11
3.5. Sheet “MoHinhTrangThai” — Mô hình trạng thái (Kripke, Chương 1) 12
Cấu trúc cột 12
Kỹ thuật liên quan 12
Cách tính / quy tắc điền 12
Ví dụ minh họa — một cụm khóa đầy đủ + một vòng đời nghiệp vụ (quanly_thuvien_muontra) 13
3.6. Sheet “ThuocTinhAnToan” — Công thức CTL cần kiểm chứng 14
Cấu trúc cột 14
Kỹ thuật liên quan 14
Cách tính / quy tắc điền 14
Ví dụ minh họa (10 dòng đầu, quanly_thuvien_muontra — đặc tả đầy đủ có 30 công thức) 14
3.7. Sheet “LoaiTruLanNhau” — Cặp mệnh đề loại trừ lẫn nhau 16

Cấu trúc cột 16
Kỹ thuật liên quan 16
Cách tính / quy tắc điền 16
Ví dụ minh họa (5 dòng đầu, quanly_thuvien_muontra) 16
3.8. Sheet “HamSinhMa” — Mô tả hàm cần sinh mã (Chương 2–4) 17
Cấu trúc cột 17
Bảng tra cứu Loai (kind) theo NgonNgu — kỹ thuật kiểm chứng tương ứng 17
Kỹ thuật liên quan 18
Cách tính / quy tắc điền 18
Ví dụ minh họa (6 dòng đầu mỗi nhóm, quanly_thuvien_muontra — đầy đủ có 30 hàm) 18
3.9. Sheet “KiemTraSoHoc” — Kiểm tra SMT / tràn số bằng Z3 (Chương 3) 20
Cấu trúc cột 20
Kỹ thuật liên quan 20
Cách tính — quy tắc chọn ngưỡng cho DieuKien (RẤT QUAN TRỌNG) 20
Ví dụ minh họa (5 dòng đầu, quanly_thuvien_muontra — đầy đủ có 30 dòng) 21
3.10. Sheet “TestCase” — Test case truy vết từ yêu cầu 22
Cấu trúc cột 22
Kỹ thuật liên quan 22
Cách tính / quy tắc điền 22
Ví dụ minh họa (5 dòng đầu, quanly_thuvien_muontra) 22
Chương 4 — Bảng tra cứu CWE mà pipeline có thể phát hiện 24
Chương 5 — Checklist hoàn thiện trước khi nộp/sử dụng 25
5.1. Cấu trúc workbook 25
5.2. Nội dung từng sheet 25
5.3. Kiểm tra bằng chạy thử 25
Chương 6 — Phụ lục: lệnh kiểm tra nhanh 27
6.1. Kiểm tra nhanh cấu trúc workbook bằng Python (không cần chạy pipeline) 27
6.2. Nạp và chạy toàn bộ pipeline, in báo cáo đầy đủ 27
6.3. Xuất ngược từ JSON sang Excel (tạo bộ khung để điền tiếp) 27
Phụ lục A — Nguồn tham khảo & phạm vi trích dẫn 28

Chương 1 — Giới thiệu và nguyên tắc chung

1.1. Mục đích của tài liệu

Tài liệu này hướng dẫn chi tiết cách tạo file đầu vào (input) cho Spec Verification Lab bằng bảng tính Excel (.xlsx), thay vì viết trực tiếp file JSON. File Excel này chính là một "đặc tả yêu cầu" (Requirement Specification) đầy đủ: sau khi nạp vào chương trình, nó được chuyển đổi thành cùng một đối tượng nội bộ Specification mà pipeline sử dụng, nên toàn bộ quy trình kiểm chứng "Đặc tả → Kripke/CTL → Sinh mã → Kiểm tra tĩnh/SMT → Fuzzing → Phân loại CWE/CVE" chạy hoàn toàn giống như khi nạp từ JSON.

Tài liệu bám sát đúng cấu trúc kỹ thuật hiện có trong mã nguồn (core/spec_excel.py, core/spec_pipeline.py, core/kripke.py, core/static_smt.py) và tài liệu định dạng gốc (dataset/specs_excel/FORMAT_EXCEL.md, dataset/specs/FORMAT.md), đồng thời minh họa lại bằng dữ liệu thực tế đã được điền và kiểm chứng thành công trong bộ đặc tả mẫu quanly_thuvien_muontra (Hệ thống Quản lý Thư viện — nghiệp vụ mượn/trả sách).

1.2. Nguyên tắc áp dụng trong tài liệu này: TẤT CẢ các mục là bắt buộc

> **BẮT BUỘC —** Trong mã nguồn gốc, chỉ sheet "ThongTin" (với trường "name") là bắt buộc về mặt kỹ thuật — các sheet còn lại có thể để trống và pipeline vẫn chạy được (phần tương ứng sẽ bị bỏ qua, chỉ cảnh báo, không báo lỗi).
>
> TUY NHIÊN, để đảm bảo một đặc tả hoàn chỉnh, có thể kiểm chứng đầy đủ cả 4 chương (Kripke/CTL, phân tích bộ nhớ tĩnh, SMT/Z3, fuzzing) và có giá trị tài liệu hóa theo đúng quy trình Requirements Engineering (IEEE 830/29148) và kiểm thử (IEEE 829), TÀI LIỆU NÀY QUY ĐỊNH: mọi sheet trong 10 sheet của workbook, và mọi cột trong mỗi sheet, đều là NỘI DUNG BẮT BUỘC phải điền — không được để trống bất kỳ sheet hay cột nào khi nộp/sử dụng đặc tả.

Cụ thể, khi áp dụng nguyên tắc "tất cả bắt buộc", người điền cần:

- Điền đủ 10/10 sheet theo đúng tên (không đổi tên sheet, không thêm/bớt sheet).
- Với mỗi sheet có cấu trúc bảng (dòng 1 là tiêu đề cột), điền ít nhất 3–5 dòng dữ liệu có ý nghĩa nghiệp vụ thực tế (không để trống, không điền dữ liệu "giả" vô nghĩa).
- Với mỗi dòng dữ liệu, điền đủ tất cả các cột được liệt kê trong tài liệu này — kể cả những cột được mã nguồn đánh dấu là "tùy chọn" (optional), vì tài liệu này coi các cột đó là bắt buộc để đặc tả đầy đủ ý nghĩa nghiệp vụ.
- Sheet MoHinhTrangThai, ThuocTinhAnToan, LoaiTruLanNhau phải khớp với nhau về mặt tên nhãn/trạng thái (xem mục 3.5–3.7).

1.3. Kiến trúc tổng thể: từ Excel đến báo cáo CWE/CVE

Workbook Excel được thiết kế theo nguyên tắc "mỗi SHEET tương ứng với MỘT LOẠI nội dung đặc tả" quen thuộc trong quy trình Requirements Engineering, thay vì gộp tất cả vào một cấu trúc JSON lồng nhau khó đọc. Quy trình xử lý gồm 4 bước:

1. Đọc workbook (parse_specification_excel trong core/spec_excel.py) — mỗi sheet được đọc thành danh sách dict (dòng 1 = tên cột, các dòng sau = dữ liệu), sau đó ánh xạ vào đối tượng Specification.

2. Kiểm chứng hình thức Chương 1 — dựng mô hình Kripke từ sheet MoHinhTrangThai, kiểm tra deadlock, kiểm tra các công thức CTL trong ThuocTinhAnToan, kiểm tra loại trừ lẫn nhau từ LoaiTruLanNhau.
3. Sinh mã & kiểm tra Chương 2–4 — sinh mã C/Python từ sheet HamSinhMa rồi quét lỗi bộ nhớ tĩnh (Chương 2), trích AST và kiểm tra SMT/tràn số bằng Z3 từ sheet KiemTraSoHoc (Chương 3), chạy black-box/concolic fuzzing trên các hàm Python (Chương 4).
4. Tổng hợp báo cáo — mọi phát hiện được ánh xạ tự động sang mã CWE chuẩn (kèm ví dụ CVE minh họa), in ra báo cáo đầy đủ theo report.to_text(), có liệt kê lại các test case truy vết từ sheet TestCase.

Chương 2 — Cấu trúc tổng thể của Workbook (10 sheet)

Workbook .xlsx phải chứa đúng 10 sheet, đặt tên chính xác (không dấu, phân biệt hoa/thường) như bảng dưới đây. Thứ tự sheet trong workbook không bắt buộc, nhưng nên giữ đúng thứ tự để dễ đối chiếu khi kiểm tra chéo.

STT	Tên sheet (chính xác)	Nội dung đặc tả	Chương liên quan
1	ThongTin	Thông tin chung: tên & mô tả hệ thống	Toàn cục
2	YeuCauChucNang	Yêu cầu chức năng + yêu cầu hình thức (an toàn/sống còn/bộ nhớ)	Ch.1–2
3	YeuCauPhiChucNang	Yêu cầu phi chức năng (hiệu năng, độ tin cậy...)	Tài liệu hóa
4	RangBuoc	Ràng buộc công nghệ/thiết kế/tài nguyên	Tài liệu hóa
5	MoHinhTrangThai	Mô hình trạng thái (state machine) cho Kripke	Chương 1
6	ThuocTinhAnToan	Công thức CTL cần kiểm chứng	Chương 1
7	LoaiTruLanNhau	Cặp mệnh đề nguyên tử kiểm tra mutual exclusion	Chương 1
8	HamSinhMa	Mô tả các hàm cần sinh mã (C/Python)	Chương 2–4
9	KiemTraSoHoc	Kiểm tra SMT/tràn số bằng Z3	Chương 3
10	TestCase	Test case truy vết từ yêu cầu (tài liệu hóa)	IEEE 829

QUY TẮC ĐỌC CHUNG — Đối với mọi sheet dạng bảng, dòng 1 luôn là DÒNG TIÊU ĐỀ CỘT (đúng chính tả như trong tài liệu này), dữ liệu bắt đầu từ dòng 2. Chương trình đọc theo TÊN cột (không theo vị trí cột), nên có thể đổi thứ tự cột nhưng KHÔNG được đổi tên cột. Dòng nào để trống hoàn toàn sẽ bị bỏ qua khi đọc.

Chương 3 — Hướng dẫn chi tiết từng sheet

Mỗi mục dưới đây trình bày theo cùng một cấu trúc 5 phần: (1) Mục đích, (2) Cấu trúc cột bắt buộc, (3) Kỹ thuật liên quan trong pipeline, (4) Cách tính / quy tắc điền dữ liệu cho đúng, và (5) Ví dụ minh họa thực tế trích từ đặc tả quanly_thuvien_muontra (Hệ thống Quản lý Thư viện) đã được kiểm chứng chạy đúng qua toàn bộ pipeline.

3.1. Sheet “ThongTin” — Thông tin chung hệ thống

BẮT BUỘC — Bắt buộc phải có; đây là sheet duy nhất mã nguồn cũng coi là bắt buộc kỹ thuật.

Mô tả tổng quan hệ thống đang được đặc tả, dùng làm tiêu đề/ngữ cảnh xuyên suốt báo cáo.

Cấu trúc cột

Bảng 2 cột, mỗi dòng dữ liệu là MỘT cặp (Truong, GiaTri) — đây là kiểu bảng “khóa–giá trị”, khác với các sheet còn lại (mỗi cột là một thuộc tính riêng).

Truong (khóa)	Bắt buộc	GiaTri (nội dung điền)
name	Có	Tên định danh của đặc tả — không dấu, không khoảng trắng (khuyến nghị dùng snake_case, vd quanly_thuvien_muontra).
description	Có (theo tài liệu này)	Mô tả hệ thống bằng 2–4 câu: hệ thống làm gì, các thành phần/tiến trình đồng thời nào tồn tại, các vấn đề kỹ thuật chính (khóa/bộ nhớ/tràn số) mà đặc tả sẽ kiểm chứng.

Kỹ thuật liên quan

Không liên quan trực tiếp kỹ thuật kiểm chứng nào — nhưng name là khóa để chương trình báo lỗi rõ ràng nếu thiếu (SpecificationError), và được in ở đầu mọi báo cáo pipeline.

Cách tính / quy tắc điền

- name: đặt tên phản ánh đúng hệ thống + nghiệp vụ chính, dùng để nhận diện file khi có nhiều đặc tả (ví dụ nhiều học sinh cùng làm đề tài tương tự).
- description: nên nêu rõ NHỮNG Gì sẽ được sinh mã/kiểm chứng (vd “khóa dùng chung giữa 2 tiến trình”, “hàm phân tích mã sách/ghi log”, “bộ đếm dung lượng dữ liệu”) để người đọc hình dung ngay phạm vi đặc tả trước khi đọc các sheet chi tiết.

Ví dụ minh họa (quanly_thuvien_muontra)

Truong	GiaTri
name	quanly_thuvien_muontra
description	He thong quan ly thu vien xu ly nghiep vu muon/tra sach: khoa quan ly ton kho sach dung chung giua 2 tien trinh (worker xu ly muon-tra sach chinh + worker ghi log giao dich), cac ham xu ly ma sach/ghi log/dong phien muon, va bo dem dung luong du lieu tai lieu so hoa da tai len.

3.2. Sheet "YeuCauChucNang" — Yêu cầu chức năng

> **BẮT BUỘC —** Bắt buộc theo tài liệu này — tối thiểu 5 dòng, bao phủ đủ các danh mục safety/liveness/memory/functional.

Đây là sheet ĐẶC BIỆT: vừa chứa yêu cầu chức năng thuần túy, vừa chứa các yêu cầu hình thức liên quan trực tiếp đến kiểm chứng Kripke/CTL (an toàn), bộ nhớ (Chương 2) — phân biệt qua cột DanhMuc.

Cấu trúc cột

Cột	Bắt buộc	Kiểu / miền giá trị	Ý nghĩa
ID	Có	Chuỗi, duy nhất, vd REQ-1	Định danh yêu cầu — dùng để TestCase truy vết tới.
MoTa	Có	Chuỗi tự nhiên (VN/EN)	Nội dung yêu cầu, câu văn đầy đủ chủ ngữ – vị ngữ.
DanhMuc	Có (theo tài liệu này)	safety | liveness | memory | functional	Phân loại yêu cầu hình thức — mặc định "functional" nếu để trống, nhưng tài liệu này YÊU CẦU điền rõ ràng cho từng dòng.
MucUuTien	Có	HIGH | MEDIUM | LOW	Mức ưu tiên (chỉ mang tính tài liệu hóa).

Kỹ thuật liên quan

- safety: yêu cầu liên quan trực tiếp tới công thức CTL an toàn (thường ánh xạ 1-1 với một dòng trong sheet ThuocTinhAnToan, ví dụ AG(!violation)).
- liveness: yêu cầu về "cuối cùng sẽ xảy ra" (thường ánh xạ dạng AF/EF), ví dụ hệ thống không deadlock, mọi giao dịch cuối cùng đều kết thúc.
- memory: yêu cầu liên quan tới lỗi bộ nhớ sẽ được kiểm tra bằng static checker Chương 2 (buffer overflow, memory-leak, double-free, use-after-free) — nên đặt tên yêu cầu khớp với tên hàm tương ứng trong sheet HamSinhMa.
- functional: yêu cầu nghiệp vụ thuần túy, không gắn trực tiếp với kỹ thuật kiểm chứng hình thức nào (chỉ tài liệu hóa).

Cách tính / quy tắc điền

Nguyên tắc chọn DanhMuc: nhìn vào NỘI DUNG kiểm chứng mà yêu cầu đó nhắm tới, không phải vào việc yêu cầu đó "nghe có vẻ quan trọng" hay không:

1. Nếu yêu cầu nói về việc "không được để 2 điều gì đó xảy ra đồng thời" / "luôn phải đúng" → DanhMuc = safety, và PHẢI có ít nhất một công thức dạng AG(...) tương ứng ở sheet ThuocTinhAnToan.
2. Nếu yêu cầu nói về việc "cuối cùng phải xảy ra" / "không được treo mãi mãi" → DanhMuc = liveness, và nên có công thức dạng AF(...)/EF(...) tương ứng.
3. Nếu yêu cầu nói về cấp phát/giải phóng bộ nhớ, sao chép dữ liệu vào buffer, con trỏ → DanhMuc = memory, và PHẢI có một hàm tương ứng trong sheet HamSinhMa (cùng ngôn ngữ c).
4. Còn lại → DanhMuc = functional.

Ví dụ minh họa (5 dòng đầu, quanly_thuvien_muontra)

ID	MoTa (rút gọn)	DanhMuc	MucUuTien
REQ-1	Worker xử lý mượn-trả chính và worker ghi log không được đồng thời giữ khóa quản lý tồn kho sách (mutual exclusion)	safety	HIGH
REQ-2	Hệ thống không được rơi vào deadlock khi tranh chấp khóa quản lý tồn kho	liveness	HIGH
REQ-3	Hàm phân tích mã sách từ chuỗi đầu vào không được gây tràn bộ đệm	memory	HIGH
REQ-4	Bộ ghi log giao dịch phải giải phóng bộ nhớ đã cấp phát cho mỗi bản ghi	memory	MEDIUM
REQ-5	Hàm đóng phiên mượn sách không được giải phóng cùng một con trỏ nhiều hơn 1 lần	memory	HIGH

(Đặc tả đầy đủ có 30 yêu cầu chức năng, bao phủ cả 4 danh mục, xem Phụ lục A.)

3.3. Sheet "YeuCauPhiChucNang" — Yêu cầu phi chức năng

BẮT BUỘC — Bắt buộc theo tài liệu này — tối thiểu 5 dòng.

Ghi lại các thuộc tính chất lượng (hiệu năng, độ tin cậy, khả năng mở rộng, khả năng bảo trì...) mà pipeline KHÔNG kiểm chứng tự động bằng kỹ thuật hình thức, nhưng vẫn cần trong một đặc tả đầy đủ và làm cơ sở viết test case thủ công.

Cấu trúc cột

Cột	Bắt buộc	Kiểu / miền giá trị	Ý nghĩa
ID	Có	Chuỗi duy nhất, vd NFR-1	Định danh — có thể được TestCase truy vết tới.
MoTa	Có	Chuỗi	Mô tả thuộc tính chất lượng, NÊN chứa một con số/ngưỡng đo được.
MucUuTien	Có	HIGH | MEDIUM | LOW	Mức ưu tiên.

Kỹ thuật liên quan

Không có kỹ thuật kiểm chứng tự động tương ứng trong pipeline — đây là nội dung THUẦN TÚY tài liệu hóa, đúng tinh thần chuẩn IEEE 29148 (yêu cầu phi chức năng luôn cần được đặc tả bằng số đo được, dù không kiểm chứng bằng mô hình hình thức).

Cách tính / quy tắc điền

- Mỗi dòng NÊN có một con số cụ thể (thời gian phản hồi tính bằng ms, thông lượng tính bằng req/s, số người dùng đồng thời...), không viết chung chung kiểu "phải nhanh", "phải ổn định".
- Phân loại phổ biến: hiệu năng (performance), độ tin cậy (reliability), khả năng mở rộng (scalability), khả năng bảo trì (maintainability), bảo mật (security) — nên rải đều ít nhất 2–3 loại trong số 5 dòng.
- Ưu tiên HIGH cho các NFR liên quan trực tiếp tới an toàn dữ liệu (vd không mất dữ liệu khi crash), MEDIUM/LOW cho các chỉ tiêu về tốc độ/khả năng mở rộng thông thường.

Ví dụ minh họa (quanly_thuvien_muontra)

ID	MoTa	MucUuTien
NFR-1	Hệ thống phải xử lý tối thiểu 500 yêu cầu mượn/trả sách/giây trên phần cứng tham chiếu	MEDIUM
NFR-2	Thời gian phản hồi trung bình cho 1 yêu cầu mượn/trả sách phải dưới 50ms	MEDIUM
NFR-3	Log giao dịch phải được ghi đồng bộ (không mất dữ liệu) kể cả khi worker ghi log bị crash giữa chừng	HIGH
NFR-4	Thời gian tìm kiếm sách trong danh mục phải dưới 1 giây với kho dữ liệu 1 triệu đầu sách	MEDIUM
NFR-5	Hệ thống phải hỗ trợ tối thiểu 200 người dùng đồng thời truy cập mà không giảm hiệu năng	HIGH

3.4. Sheet “RangBuoc” — Ràng buộc hệ thống

BẮT BUỘC — Bắt buộc theo tài liệu này — tối thiểu 5 dòng.

Ràng buộc về công nghệ, thiết kế, tài nguyên áp dụng lên toàn hệ thống hoặc dự án — những giới hạn KHÔNG được vi phạm khi sinh mã/triển khai, dù không kiểm chứng tự động.

Cấu trúc cột

Cột	Bắt buộc	Kiểu / miền giá trị	Ý nghĩa
ID	Có	Chuỗi duy nhất, vd CON-1	Định danh — có thể được TestCase truy vết tới.
MoTa	Có	Chuỗi	Nội dung ràng buộc.
MucUuTien	Có	HIGH | MEDIUM | LOW	Mức ưu tiên.

Kỹ thuật liên quan

Không kiểm chứng tự động — nhưng các ràng buộc về ĐỊNH DẠNG/ĐỘ DÀI dữ liệu (vd “mã sách tối đa 16 ký tự”) chính là căn cứ để chọn giá trị KichThuocBuffer khi khai báo hàm buffer_copy ở sheet HamSinhMa (mục 3.8) — PHẢI nhất quán giữa hai sheet này.

Cách tính / quy tắc điền

- Ràng buộc công nghệ: ngôn ngữ/thư viện bắt buộc hoặc bị cấm dùng (vd “chỉ dùng thư viện chuẩn C”).
- Ràng buộc thiết kế: loại cơ chế đồng bộ hóa bắt buộc/bị cấm (vd “khóa phải là spinlock/futex, không dùng khóa cấp hệ điều hành nặng”).
- Ràng buộc dữ liệu: độ dài tối đa, bảng mã ký tự, định dạng chuẩn (vd ISBN-13) — các con số này PHẢI khớp với KichThuocBuffer tương ứng ở sheet HamSinhMa.
- Ràng buộc tài nguyên: giới hạn phần cứng/bộ nhớ nếu có.

Ví dụ minh họa (quanly_thuvien_muontra) — chú ý cột MoTa khớp KichThuocBuffer ở mục 3.8

ID	MoTa	MucUuTien
CON-1	Khóa quản lý tồn kho phải là mutex nhẹ (spinlock/futex), không được dùng khóa cấp hệ điều hành nặng	MEDIUM
CON-2	Mã sách tối đa 16 ký tự, mã hóa ASCII	HIGH
CON-3	Mã mở khóa tủ sách phải là cặp số nguyên 16-bit không dấu	MEDIUM
CON-4	Mã độc giả tối đa 12 ký tự, chỉ gồm chữ số	HIGH
CON-5	Mã ISBN phải tuân theo định dạng ISBN-13 (13 chữ số)	HIGH

3.5. Sheet "MoHinhTrangThai" — Mô hình trạng thái (Kripke, Chương 1)

BẮT BUỘC — Bắt buộc theo tài liệu này — mô hình phải KHÔNG có trạng thái deadlock.

Mỗi DÒNG mô tả MỘT trạng thái của hệ thống. Toàn bộ các dòng cùng nhau tạo thành một cấu trúc Kripke (states, initial, labeling, transitions) — nền tảng để kiểm chứng công thức CTL ở sheet ThuocTinhAnToan.

Cấu trúc cột

Cột	Bắt buộc	Kiểu / miền giá trị	Ý nghĩa
TrangThai	Có	Chuỗi duy nhất (tên trạng thái)	Định danh trạng thái, dùng làm mốc tham chiếu trong ChuyenDenCacTrangThai của các dòng khác và trong công thức CTL.
KhoiTao	Có (đánh dấu rõ ràng: "x" hoặc để trống)	"x" hoặc rỗng	Đánh dấu "x" nếu đây là trạng thái khởi tạo. Có thể có NHIỀU trạng thái khởi tạo (đánh dấu "x" ở nhiều dòng).
Nhan	Có	Danh sách nhãn, phân cách dấu phẩy	Các mệnh đề nguyên tử (atomic propositions) đúng tại trạng thái này — dùng trong công thức CTL.
ChuyenDenCacTrangThai	Có	Danh sách tên trạng thái, phân cách dấu phẩy	Các trạng thái có thể chuyển TỚI từ trạng thái này (mỗi giá trị sinh 1 cặp chuyển tiếp).

Kỹ thuật liên quan

- Cấu trúc Kripke (core/kripke.py): tập trạng thái + hàm gán nhãn + quan hệ chuyển tiếp — nền tảng lý thuyết model checking.
- Kiểm tra deadlock (find_deadlock_states): MỌI trạng thái phải có ÍT NHẤT 1 dịch chuyển đi ra, nếu không sẽ bị báo là deadlock state.
- Model checking CTL (CTLChecker.check ở sheet 3.6): duyệt đồ thị trạng thái để xác định công thức đúng/sai.

Cách tính / quy tắc điền

1. Mô hình hóa mỗi TÀI NGUYÊN DÙNG CHUNG (khóa/mutex) giữa các tiến trình bằng một cụm 4 trạng thái theo khuôn mẫu chuẩn: <ten>_free (trạng thái rảnh, khởi tạo, nhãn "safe") → <ten>_main_holds / <ten>_log_holds (một bên giữ khóa) → <ten>_conflict (CẢ HAI cùng giữ khóa — đây là trạng thái LỖI cố ý, nhãn gồm cả "violation") → quay lại free.
2. Đặt nhãn "violation" (hoặc <prefix>_violation nếu có nhiều cụm khóa) cho đúng trạng thái vi phạm — đây là QUY ƯỚC (không bắt buộc kỹ thuật) nhưng cần thiết để công thức AG(!violation) trong sheet 3.6 có ý nghĩa kiểm tra đúng.
3. Với vòng đời nghiệp vụ (không phải khóa) — vd "sách đang mượn → đã trả / hư hỏng / thất lạc" — mô hình hóa thành chuỗi trạng thái tuyến tính hoặc rẽ nhánh, TRẠNG THÁI CUỐI (kết thúc vòng đời) LÀ TRƯỜNG HỢP ĐẶC BIỆT: nếu để ChuyenDenCacTrangThai trống, trạng thái đó sẽ bị công cụ báo là "deadlock". Nếu đó là kết thúc hợp lệ (không phải lỗi), hãy thêm một dịch chuyển VÒNG

LẶP về chính nó (tự trỏ tới TrangThai của chính dòng đó) để thể hiện "trạng thái ổn định cuối cùng" mà không bị tính là deadlock.

4. Nếu hệ thống có N tiến trình/thành phần đồng thời độc lập cùng tranh chấp một loại tài nguyên tương tự nhau (ví dụ 8 cặp worker chính/log khác nhau trong ví dụ thư viện), lặp lại đúng khuôn mẫu 4 trạng thái ở bước 1 cho từng cặp, đổi tiền tố tên trạng thái/nhãn (vd tkdg_, hddt_, qp_...) để không trùng tên.

Ví dụ minh họa — một cụm khóa đầy đủ + một vòng đời nghiệp vụ (quanly_thuvien_muontra)

TrangThai	KhoiTao	Nhan	ChuyenDenCacTrangThai
free	x	safe	main_holds, log_holds
main_holds		main_locked	free, conflict
log_holds		log_locked	free
conflict		main_locked, log_locked, violation	free
sach_con_trong	x	available	sach_dang_muon
sach_dang_muon		on_loan	sach_con_trong, sach_hu_hong, sach_that_lac
sach_hu_hong		damaged	sach_con_trong
sach_that_lac		lost	sach_that_lac

Lưu ý dòng cuối: sach_that_lac (sách thất lạc) là trạng thái kết thúc vòng đời hợp lệ, KHÔNG PHẢI lỗi — nên được cho tự trỏ về chính nó (sach_that_lac) ở cột ChuyenDenCacTrangThai để tránh bị báo deadlock. Đặc tả đầy đủ mô hình hóa 8 cụm khóa tương tự (main/log) cho 8 module nghiệp vụ khác nhau và khoảng 15 vòng đời nghiệp vụ khác, tổng cộng 59 trạng thái — xem Phụ lục A.

3.6. Sheet "ThuocTinhAnToan" — Công thức CTL cần kiểm chứng

> **BẮT BUỘC —** Bắt buộc theo tài liệu này — mỗi cụm khóa/vòng đời ở sheet 3.5 phải có ít nhất 1 công thức tương ứng.

Mỗi dòng là MỘT công thức logic thời gian nhánh (CTL — Computation Tree Logic) sẽ được kiểm tra trên mô hình Kripke dựng từ sheet MoHinhTrangThai.

Cấu trúc cột

Cột	Bắt buộc	Kiểu / miền giá trị	Ý nghĩa
CongThucCTL	Có	Cú pháp CTL hỗ trợ (xem bên dưới)	Công thức cần kiểm chứng đúng/sai trên mô hình.

Kỹ thuật liên quan

Toán tử CTL được hỗ trợ bởi CTLChecker (core/kripke.py):

Toán tử	Ý nghĩa (đọc là)	Cách dùng phổ biến
AG(p)	"Luôn luôn, trên mọi nhánh" — p đúng ở MỌI trạng thái đạt được	Thuộc tính an toàn: AG(!violation) — không bao giờ vào trạng thái vi phạm.
EF(p)	"Tồn tại một nhánh mà cuối cùng" p đúng	Kiểm tra tính khả đạt (reachability): trạng thái p có thể xảy ra.
AF(p)	"Trên mọi nhánh, cuối cùng" p đúng	Thuộc tính sống còn (liveness): mọi tiến trình rồi sẽ đạt p.
EG(p)	"Tồn tại một nhánh mà p đúng mãi mãi"	Kiểm tra khả năng "kẹt mãi" ở trạng thái thỏa p.
AX(p)	"Trên mọi nhánh, ở bước kế tiếp" p đúng	Ràng buộc cục bộ 1 bước.
EX(p)	"Tồn tại một nhánh mà bước kế tiếp" p đúng	Kiểm tra khả năng chuyển tiếp.
!p, p & q, p | q, p -> q	Phủ định / hội / tuyển / kéo theo đơn giản	Kết hợp nhiều nhãn, vd AG(sach_dang_muon -> AF(sach_con_trong | sach_that_lac)).

Cách tính / quy tắc điền

1. Với mỗi cụm khóa (mục 3.5): thêm CHÍNH XÁC 2 công thức — AG(!violation) (không bao giờ vi phạm loại trừ lẫn nhau) và EF(main_locked) (trạng thái giữ khóa vẫn có thể đạt tới, chứng minh mô hình không tầm thường/không khóa chết ngay từ đầu).
2. Với mỗi vòng đời nghiệp vụ có trạng thái "đang xử lý": thêm công thức dạng AG(<trang_thai_dang_xu_ly> -> AF(<mot_trong_cac_trang_thai_ket_thuc>)) để khẳng định mọi tiến trình xử lý cuối cùng đều kết thúc (không treo mãi).
3. Đặt tên các nhãn trong công thức CTL PHẢI khớp CHÍNH XÁC (không dấu, phân biệt hoa/thường) với nhãn đã khai báo ở cột Nhan của sheet MoHinhTrangThai — sai chính tả sẽ khiến công thức luôn được đánh giá sai/lỗi.
4. Không viết công thức trùng lặp hoàn toàn; mỗi công thức nên kiểm tra một khía cạnh khác nhau (an toàn, khả đạt, hoặc sống còn).

Ví dụ minh họa (10 dòng đầu, quanly_thuvien_muontra — đặc tả đầy đủ có 30 công thức)

#	CongThucCTL	Loại
1	AG(!violation)	An toàn (mutex chính)
2	EF(main_locked)	Khả đạt (mutex chính)
3	AG(!tkdg_violation)	An toàn (mutex module 2)
4	EF(tkdg_main_locked)	Khả đạt (mutex module 2)
5	AG(sach_dang_muon -> AF(sach_con_trong | sach_that_lac))	Sống còn (vòng đời mượn sách)
6	AG(dat_truoc_dang_cho -> AF(dat_truoc_hoan_tat | dat_truoc_het_han))	Sống còn (vòng đời đặt trước)
7	AG(phien_muon_dang_mo -> AF(phien_muon_da_dong))	Sống còn (phiên mượn)
8	AG(!(sach_con_trong & sach_dang_muon))	An toàn (loại trừ trạng thái mâu thuẫn)
9	EF(dat_truoc_hoan_tat)	Khả đạt (đặt trước hoàn tất)
10	AG(tai_khoan_bi_khoa -> EF(tai_khoan_hoat_dong))	Khả năng phục hồi (tài khoản bị khóa có thể mở lại)

3.7. Sheet "LoaiTruLanNhau" — Cặp mệnh đề loại trừ lẫn nhau

> **BẮT BUỘC —** Bắt buộc theo tài liệu này — mỗi cụm khóa ở sheet 3.5 phải có đúng 1 dòng tương ứng.

Ghi lại các cặp mệnh đề nguyên tử KHÔNG ĐƯỢC cùng đúng tại một trạng thái có thể đạt tới từ trạng thái khởi tạo — tương đương kiểm tra công thức AG(!(p1 & p2)).

Cấu trúc cột

Cột	Bắt buộc	Kiểu / miền giá trị	Ý nghĩa
MenhDe1	Có	Tên một nhãn đã khai báo ở sheet MoHinhTrangThai	Mệnh đề nguyên tử thứ nhất.
MenhDe2	Có	Tên một nhãn đã khai báo ở sheet MoHinhTrangThai	Mệnh đề nguyên tử thứ hai — không được cùng đúng với MenhDe1.

> **LƯU Ý KỸ THUẬT —** Trường mutual_exclusion gốc trong Specification chỉ nhận ĐÚNG MỘT cặp (atom1, atom2) — khi đọc từ Excel, hàm parse_specification_excel() CHỈ LẤY DÒNG ĐẦU TIÊN của sheet này để tạo kiểm tra mutex chính thức trong verify_specification(). Các dòng còn lại vẫn PHẢI được điền đầy đủ theo yêu cầu "tất cả bắt buộc" của tài liệu này (đóng vai trò tài liệu hóa đầy đủ các cặp loại trừ trong hệ thống), nhưng chỉ dòng đầu tiên được pipeline TỰ ĐỘNG kiểm tra bằng Z3/CTL — vì vậy hãy đặt cặp mệnh đề QUAN TRỌNG NHẤT (ứng với cụm khóa chính của hệ thống) ở DÒNG ĐẦU TIÊN.

Kỹ thuật liên quan

Kiểm tra mutual exclusion trong verify_specification() (core/spec_pipeline.py) — duyệt toàn bộ trạng thái đạt được từ trạng thái khởi tạo, báo lỗi mutual-exclusion-violation (CWE-667) nếu tìm thấy trạng thái mà cả hai nhãn cùng có mặt.

Cách tính / quy tắc điền

1. Dòng đầu tiên: LUÔN là cặp nhãn "<khóa>_locked" ứng với cụm khóa CHÍNH của hệ thống (ví dụ main_locked / log_locked) — đây là cặp được pipeline kiểm tra tự động.
2. Các dòng tiếp theo: liệt kê đầy đủ các cặp loại trừ khác trong hệ thống để tài liệu hóa (các cụm khóa phụ, hoặc các trạng thái nghiệp vụ không thể cùng tồn tại như "violation" và "safe", hoặc "sach_con_trong" và "sach_dang_muon").
3. Mỗi cặp phải là hai nhãn PHÂN BIỆT, cả hai đều đã xuất hiện trong cột Nhan của sheet MoHinhTrangThai.

Ví dụ minh họa (5 dòng đầu, quanly_thuvien_muontra)

MenhDe1	MenhDe2
main_locked	log_locked
tkdg_main_locked	tkdg_log_locked
violation	safe
sach_con_trong	sach_dang_muon
tai_khoan_hoat_dong	tai_khoan_bi_khoa

3.8. Sheet "HamSinhMa" — Mô tả hàm cần sinh mã (Chương 2–4)

> **BẮT BUỘC —** Bắt buộc theo tài liệu này — mỗi yêu cầu DanhMuc=memory ở sheet 3.2 phải có 1 hàm C tương ứng; nên có ít nhất 1 hàm Python kind=divide và 1 hàm kind=magic_value.

Mỗi dòng mô tả MỘT hàm sẽ được SINH MÃ NGUỒN tự động (C hoặc Python) rồi đưa qua các công cụ kiểm tra tương ứng: quét lỗi bộ nhớ tĩnh (hàm C, Chương 2), trích AST cho SMT (hàm arithmetic, Chương 3), hoặc fuzzing (hàm Python, Chương 4).

Cấu trúc cột

Cột	Bắt buộc	Kiểu / miền giá trị	Ý nghĩa
Ten	Có	Tên hàm hợp lệ (định danh)	Tên hàm được sinh ra.
NgonNgu	Có	c | python	Ngôn ngữ sinh mã.
Loai	Có	Xem bảng "tra cứu Loai" bên dưới	Loại hàm — quyết định khuôn mẫu mã được sinh.
ThamSo	Có	Danh sách tên tham số, phân cách dấu phẩy (để trống nếu Loai không cần)	Tham số đầu vào của hàm.
KichThuocBuffer	Có (mặc định 16 nếu để trống)	Số nguyên dương	Chỉ dùng cho buffer_copy/alloc_release — kích thước buffer/vùng cấp phát.
Loi	Có — ghi rõ ràng buộc hoặc để trống có chủ đích	để trống | buffer-overflow | memory-leak | double-free | use-after-free	Loại lỗi CỐ Ý cấy vào mã sinh ra (chỉ áp dụng ngôn ngữ c) — để trống nghĩa là hàm KHÔNG lỗi (dùng strncpy an toàn).
GiaTriBiMat1	Có, khi Loai=magic_value; để trống các trường hợp khác	Số nguyên	Giá trị bí mật thứ nhất (chỉ dùng kind=magic_value, Python).
GiaTriBiMat2	Có, khi Loai=magic_value; để trống các trường hợp khác	Số nguyên	Giá trị bí mật thứ hai (chỉ dùng kind=magic_value, Python).

Bảng tra cứu Loai (kind) theo NgonNgu — kỹ thuật kiểm chứng tương ứng

NgonNgu	Loai	Mã sinh ra làm gì	Kỹ thuật kiểm tra
c	buffer_copy	Sao chép chuỗi vào buffer kích thước cố định (KichThuocBuffer). Loi=buffer-overflow → dùng strcpy() (không an toàn); để trống → dùng strncpy() (an toàn).	Static memory analyzer (Chương 2) → CWE-120 nếu có lỗi.
c	alloc_release	Sinh cặp malloc()/free(). Loi có thể là memory-leak (không free), double-free (free 2 lần), use-after-free (dùng sau khi free), hoặc để trống (không lỗi).	Static memory analyzer (Chương 2) → CWE-401/415/416.
c	arithmetic	Sinh hàm int ten(int a, int b) { return a + b; } từ ThamSo.	Trích xuất AST (Chương 3) làm đầu vào cho phân tích SMT.

NgonNgu	Loai	Mã sinh ra làm gì	Kỹ thuật kiểm tra
python	divide	Sinh def ten(a, b): return a // b.	Black-box fuzzer (Chương 4) → tìm crash ZeroDivisionError.
python	magic_value	Sinh hàm có nhánh điều kiện lồng nhau: if a == m1: if b == m2: raise ... (m1, m2 lấy từ GiaTriBiMat1/2).	Concolic/white-box fuzzer (Chương 4) — kiểm tra khả năng thuật toán "đoán mò" (black-box) khó tìm ra nhánh lỗi so với concolic.

Kỹ thuật liên quan

- Hàm C được gộp lại và quét bằng core.memory_analyzer.check_c_source() (Chương 2), đồng thời trích AST bằng core.static_smt.extract_c_statements() (Chương 3).
- Hàm Python được biên dịch an toàn (chỉ hỗ trợ một tập con cú pháp giới hạn) rồi chạy qua blackbox_fuzz()/concolic_fuzz() (Chương 4).

Cách tính / quy tắc điền

1. Với mỗi ràng buộc độ dài dữ liệu đã khai báo ở sheet RangBuoc (mục 3.4) — vd "mã sách tối đa 16 ký tự" — tạo MỘT hàm buffer_copy có KichThuocBuffer bằng đúng con số đó (ở đây là 16), tên hàm nên bắt đầu bằng parse_ hoặc handle_ để phản ánh vai trò phân tích dữ liệu đầu vào.
2. Với mỗi thao tác "ghi log"/"lưu bản ghi động" trong nghiệp vụ → tạo một hàm alloc_release, Loi=memory-leak, để minh họa lỗi quên giải phóng bộ nhớ điển hình khi xử lý luồng dữ liệu liên tục.
3. Với mỗi thao tác "đóng phiên/kết thúc giao dịch" → tạo một hàm alloc_release, Loi=double-free, minh họa lỗi gọi giải phóng tài nguyên nhiều lần khi có nhiều luồng xử lý cùng gọi hàm đóng phiên.
4. Với mỗi thao tác "giải phóng bộ đệm rồi vẫn còn tham chiếu dùng lại" (cache, hàng đợi) → tạo một hàm alloc_release, Loi=use-after-free.
5. Với mỗi tỉ lệ/tỷ số nghiệp vụ có MẪU SỐ có thể bằng 0 (vd tỉ lệ sách đã trả = số trả / số mượn, khi chưa có sách nào được mượn) → tạo một hàm kind=divide (Python) để fuzzer chứng minh được rằng đầu vào biên (mẫu số = 0) gây crash ZeroDivisionError.
6. Với mỗi "mã bí mật" nghiệp vụ dạng cặp số (mã mở khóa tủ, PIN, mã ghi đè của nhân viên) → tạo một hàm kind=magic_value (Python), GiaTriBiMat1/2 là 2 số nguyên BẤT KỲ (nên khác giữa các hàm để tránh trùng lặp), minh họa lỗi "kiểm tra giá trị bí mật bằng nhánh if lồng nhau" rất khó bị black-box fuzzer tìm ra ngẫu nhiên nhưng concolic fuzzer tìm ra nhanh.
7. Với mỗi phép toán số học đơn giản cần trích AST cho Chương 3 → tạo một hàm kind=arithmetic (C), 2 tham số trở lên.

Ví dụ minh họa (6 dòng đầu mỗi nhóm, quanly_thuvien_muontra — đầy đủ có 30 hàm)

Ten	NgonNgu	Loai	ThamSo	Buffer	Loi
parse_book_id	c	buffer_copy	raw_id	16	buffer-overflow
parse_reader_id	c	buffer_copy	raw_id	12	buffer-overflow
write_transaction_log	c	alloc_release		32	memory-leak
close_borrow_session	c	alloc_release		4	double-free

Ten	NgonNgu	Loai	ThamSo	Buffer	Loi
release_borrow_session_buffer	c	alloc_release		8	use-after-free
return_rate_ratio	python	divide	returned, borrowed	16	
locker_unlock_code	python	magic_value	code_a, code_b	16	

Với 2 dòng cuối (Python), cột GiaTriBiMat1/GiaTriBiMat2 tương ứng là (return_rate_ratio: không dùng, để trống vì kind=divide) và (locker_unlock_code: 6284 / 4917).

3.9. Sheet “KiemTraSoHoc” — Kiểm tra SMT / tràn số bằng Z3 (Chương 3)

> **BẮT BUỘC —** Bắt buộc theo tài liệu này — tối thiểu 5 dòng, mỗi dòng khai báo cả DieuKien và BieuThucTran.

Mỗi dòng mô tả MỘT kiểm tra số học sẽ được giải bằng bộ giải ràng buộc Z3 (SMT solver): kiểm tra tính THỎA ĐƯỢC (satisfiability) của một điều kiện logic, và/hoặc khả năng TRÀN SỐ (overflow) của một biểu thức số học.

Cấu trúc cột

Cột	Bắt buộc	Kiểu / miền giá trị	Ý nghĩa
Ten	Có	Tên định danh	Tên bộ đếm/biến nghiệp vụ đang kiểm tra.
BienSoNguyen	Có	Danh sách tên biến, phân cách dấu phẩy	Khai báo các biến SỐ NGUYÊN sẽ dùng trong DieuKien/BieuThucTran.
DieuKien	Có	Biểu thức logic Python-like (so sánh, and/or)	Điều kiện kiểm tra SAT/UNSAT bằng check_satisfiability().
BieuThucTran	Có	Biểu thức số học thuần túy (chỉ + − *, KHÔNG so sánh)	Biểu thức kiểm tra khả năng tràn int32 bằng check_int32_overflow().

Kỹ thuật liên quan

check_satisfiability(condition, int_vars): dựng công thức Z3 từ DieuKien, gọi solver.check() — trả về “sat” (tồn tại bộ giá trị thỏa mãn, kèm mô hình minh họa) hoặc “unsat” (không tồn tại).

check_int32_overflow(expr, int_vars): dựng biến Z3 kiểu Int cho mỗi biến, RÀNG BUỘC mỗi biến nằm trong đúng phạm vi số nguyên có dấu 32-bit [-2^{31}, $2^{31}-1$] = [-2147483648, 2147483647] (đây là ràng buộc CỐ ĐỊNH của công cụ, áp dụng cho MỌI biến, không phụ thuộc vào giá trị bạn viết trong cột DieuKien), sau đó kiểm tra xem BieuThucTran có thể nằm NGOÀI phạm vi đó hay không. Kết quả “sat” nghĩa là CÓ THỂ tràn số.

Cách tính — quy tắc chọn ngưỡng cho DieuKien (RẤT QUAN TRỌNG)

Vì check_int32_overflow() luôn cho các biến dao động tự do trong toàn bộ phạm vi int32, TỔNG của hai biến int32 hầu như LUÔN có thể vượt phạm vi int32 (ví dụ 2 tỷ + 2 tỷ) — nên bản thân việc “tràn được” gần như luôn đúng về mặt toán học thuần túy. Ý nghĩa THỰC TIỄN của cột DieuKien không nằm ở việc “làm cho tràn xảy ra”, mà ở việc CHỨNG MINH RẰNG TRÀN SỐ VẪN CÓ THỂ XẢY RA NGAY CẢ KHI GIỚI HẠN LẠI THEO Ý NGHĨA NGHIỆP VỤ THỰC TẾ của biến đó (ví dụ một bộ đếm được lập trình viên NGỠ RẰNG chỉ cần kiểu 16-bit là đủ). Quy tắc chọn ngưỡng:

1. Xác định Ý NGHĨA NGHIỆP VỤ của từng biến (bộ đếm hằng ngày, tổng lũy kế, dung lượng byte...) và kiểu dữ liệu mà lập trình viên có thể LẦM TƯỞNG là đủ dùng cho biến đó trong thực tế (vd đếm số sách mượn/ngày tưởng chỉnh int16 là đủ).
2. Tra ngưỡng trên (giá trị tối đa biểu diễn được) của kiểu đó theo bảng sau, và viết điều kiện <bien1> >= 0 and <bien2> >= 0 and <bien1> + <bien2> > <nguong> trong cột DieuKien — ĐÂY LÀ PHÉP TÍNH bạn cần thực hiện thủ công cho mỗi dòng:

Kiểu số giả định (ý nghĩa nghiệp vụ)	Công thức tính ngưỡng trên	Giá trị ngưỡng
int16 có dấu (đếm hằng ngày nhỏ: số sách mượn/ngày, số quá hạn...)	$2^{15} - 1$	32767
uint16 không dấu (đếm tích lũy vừa: độc giả đang hoạt động, số lượt đặt trước...)	$2^{16} - 1$	65535
int32 có dấu (đếm/dung lượng lớn: số byte tài liệu số, số tiền phạt VND, số dòng báo cáo...)	gần $2^{31} - 1$ (dùng ngưỡng tròn ~2 tỷ để chắc chắn vượt biên an toàn)	2 000 000 000
uint32 không dấu (dung lượng sao lưu/hệ thống lớn nhất)	$2^{32} - 1$	4 294 967 295

3. Cột BieuThucTran luôn là chính biểu thức tổng của các biến trong DieuKien (vd bien1 + bien2) — không viết lại điều kiện so sánh ở đây (chỉ + − *), vì đây là biểu thức được đưa cho check_int32_overflow() để kiểm tra tràn theo phạm vi int32 CỐ ĐỊNH của công cụ, độc lập với ngưỡng nghiệp vụ đã viết ở DieuKien.
4. Kết quả mong đợi khi chạy: DieuKien luôn cho kết quả "sat" (ngưỡng nghiệp vụ vẫn nằm trong phạm vi int32 nên luôn tồn tại được), còn BieuThucTran cũng cho "sat" (tổng 2 biến int32 luôn có thể vượt phạm vi int32) — hai kết quả này cùng nhau MINH HỌA rằng: dù ngưỡng nghiệp vụ nhỏ hơn nhiều so với giới hạn int32, lập trình viên vẫn không nên chủ quan chọn kiểu dữ liệu hẹp (int16/uint16) cho các phép cộng dồn, vì tổng vẫn có thể vượt khỏi kiểu hẹp đó rất nhanh trong thực tế.

Ví dụ minh họa (5 dòng đầu, quanly_thuvien_muontra — đầy đủ có 30 dòng)

Ten	BienSoNguyen	Ngưỡng (DieuKien)	BieuThucTran
digitized_document_byte_counter	uploaded_bytes, pending_bytes	> 2 000 000 000	uploaded_bytes + pending_bytes
total_borrowed_books_counter	borrowed_today, borrowed_backlog	> 32 767	borrowed_today + borrowed_backlog
total_returned_books_counter	returned_today, returned_backlog	> 32 767	returned_today + returned_backlog
total_fine_amount_counter	fine_collected_vnd, fine_pending_vnd	> 2 000 000 000	fine_collected_vnd + fine_pending_vnd
total_registered_readers_counter	active_readers, suspended_readers	> 65 535	active_readers + suspended_readers

3.10. Sheet "TestCase" — Test case truy vết từ yêu cầu

> **BẮT BUỘC —** Bắt buộc theo tài liệu này — tối thiểu 5 dòng, mỗi dòng truy vết tới ít nhất 1 yêu cầu ở sheet YeuCauChucNang/YeuCauPhiChucNang/RangBuoc.

Sheet CHỈ MANG TÍNH TÀI LIỆU HÓA / BÁO CÁO — run_pipeline() KHÔNG tự động "chạy" các test case này (chỉ liệt kê lại trong báo cáo, mục "[Test Case] Truy vết từ đặc tả"). Mục đích là rèn kỹ năng viết test case truy vết từ đặc tả yêu cầu, gắn với cấu trúc tài liệu kiểm thử IEEE 829 (Test Case Specification).

Cấu trúc cột

Cột	Bắt buộc	Kiểu / miền giá trị	Ý nghĩa
ID	Có	Chuỗi duy nhất, vd TC-1	Định danh test case.
YeuCauLienQuan	Có	Danh sách ID yêu cầu, phân cách dấu phẩy	Truy vết tới REQ-.../NFR-.../CON-... — PHẢI tồn tại ở các sheet 3.2–3.4.
TieuDe	Có	Chuỗi ngắn gọn	Tên/tóm tắt test case.
DieuKienTienQuyet	Có	Chuỗi	Trạng thái/điều kiện hệ thống trước khi thực hiện test.
CacBuoc	Có	Mỗi bước 1 dòng trong cùng 1 ô (Alt+Enter)	Các bước thực hiện tuần tự.
KetQuaMongDoi	Có	Chuỗi	Kết quả kỳ vọng, nên nêu rõ công thức/lệnh kiểm tra tương ứng.
MucUuTien	Có	HIGH | MEDIUM | LOW	Mức ưu tiên.

Kỹ thuật liên quan

test_cases_report() (core/spec_pipeline.py) in danh sách test case kèm truy vết requirement_ids — liên hệ trực tiếp cấu trúc tài liệu kiểm thử IEEE 829, nội dung được giới thiệu ở Chương 8 (CSE703184).

Cách tính / quy tắc điền

1. Với mỗi cặp cụm khóa (mục 3.5–3.7): viết 1 test case truy vết tới cặp yêu cầu safety+liveness tương ứng (vd REQ-1, REQ-2), bước thực hiện mô tả kịch bản 2 tiến trình cùng xin khóa, kết quả mong đợi nêu rõ công thức CTL cần đúng (vd "AG(!violation) ĐÚNG và không có trạng thái deadlock").
2. Với mỗi hàm C có Loi khác rỗng (mục 3.8): viết 1 test case, DieuKienTienQuyet nêu tên hàm + Loi đã khai báo, bước thực hiện là "sinh mã C cho hàm → chạy check_c_source()", kết quả mong đợi nêu rõ mã CWE dự kiến phát hiện được (CWE-120/401/415/416).
3. Với mỗi hàm kind=divide (Python): viết 1 test case, bước thực hiện "chạy blackbox_fuzz() N lần", kết quả mong đợi "tìm được ít nhất 1 crash ZeroDivisionError".
4. Với mỗi dòng KiemTraSoHoc: viết 1 test case, bước thực hiện "chạy check_int32_overflow() bằng Z3 trên BieuThucTran", kết quả mong đợi "kết quả SAT (có thể tràn số)".

Ví dụ minh họa (5 dòng đầu, quanly_thuvien_muontra)

ID	YeuCauLienQuan	TieuDe	KetQuaMongDoi	Ưu tiên
TC-1	REQ-1, REQ-2	Worker muon-tra chinh va worker log tranh chap khoa ton kho	AG(!violation) ĐÚNG và không có trạng thái deadlock	HIGH
TC-2	REQ-3	Mã sách vượt quá 16 ký tự	Phát hiện 'dangerous-function' (CWE-120)	HIGH
TC-3	REQ-4, REQ-5, REQ-6	Kiểm tra vòng đời bộ nhớ của log/phiên mượn	Phát hiện đủ 3 loại lỗi: memory-leak (CWE-401), double-free (CWE-415), use-after-free (CWE-416)	HIGH
TC-4	REQ-7	Bộ đếm byte tài liệu số hóa vượt giới hạn int32	Kết quả SAT (có thể tràn số)	MEDIUM
TC-5	REQ-8	Tính tỉ lệ sách đã trả với borrowed=0	Tìm được ít nhất 1 crash ZeroDivisionError	MEDIUM

Chương 4 — Bảng tra cứu CWE mà pipeline có thể phát hiện

Mọi phát hiện từ Chương 1–4 (vi phạm CTL, mutex, deadlock, các loại lỗi bộ nhớ, tràn số, crash khi fuzzing) đều được run_pipeline() ánh xạ tự động sang một mã CWE (Common Weakness Enumeration) chuẩn kèm ví dụ CVE minh họa (nếu có). Bảng dưới đây giúp đối chiếu: khi điền một giá trị ở cột Loi (sheet HamSinhMa) hay một công thức CTL (sheet ThuocTinhAnToan), người điền cần biết trước NÓ SẼ tương ứng với CWE nào trong báo cáo.

Nguồn phát hiện	Mã CWE	Tên	Xuất hiện khi nào trong template Excel
buffer-overflow (Loi, HamSinhMa)	CWE-120	Buffer Copy without Checking Size of Input	Loai=buffer_copy, Loi=buffer-overflow
memory-leak (Loi, HamSinhMa)	CWE-401	Missing Release of Memory after Effective Lifetime	Loai=alloc_release, Loi=memory-leak
double-free (Loi, HamSinhMa)	CWE-415	Double Free	Loai=alloc_release, Loi=double-free
use-after-free (Loi, HamSinhMa)	CWE-416	Use After Free	Loai=alloc_release, Loi=use-after-free
Tràn số (BieuThucTran, KiemTraSoHoc)	CWE-190	Integer Overflow or Wraparound	check_int32_overflow() trả về “sat”
Deadlock (MoHinhTrangThai)	CWE-833	Deadlock	Một trạng thái không có ChuyenDenCacTrangThai (không tự trỏ về chính nó)
Vi phạm mutex (LoaiTruLanNhau)	CWE-667	Improper Locking	Cặp mệnh đề ở dòng đầu tiên sheet LoaiTruLanNhau cùng đúng tại 1 trạng thái đạt được
Vi phạm công thức CTL (ThuocTinhAnToan)	CWE-693	Protection Mechanism Failure	Một công thức trong sheet ThuocTinhAnToan được đánh giá là sai
Crash khi fuzzing (HamSinhMa, Python)	CWE-248	Uncaught Exception	blackbox_fuzz()/concolic_fuzz() tìm được đầu vào gây lỗi (ZeroDivisionError hoặc raise trong magic_value)

MẸO ĐỌC BÁO CÁO — Vì tài liệu này yêu cầu điền ĐẦY ĐỦ các trường hợp lỗi (buffer-overflow, memory-leak, double-free, use-after-free ở sheet HamSinhMa, và ít nhất 1 cụm khóa cố ý sai — dòng “conflict” ở mục 3.5), báo cáo cuối cùng SẼ liệt kê đủ các mã CWE trên — đây là hành vi ĐÚNG THIẾT KẾ (mục tiêu là luyện tập phát hiện lỗi, không phải tạo ra một hệ thống “sạch tuyệt đối”).

Chương 5 — Checklist hoàn thiện trước khi nộp/sử dụng

BẮT BUỘC — Tất cả các mục dưới đây đều phải được đánh dấu ĐẠT trước khi coi file Excel là hoàn chỉnh.

5.1. Cấu trúc workbook

- ☐ Đủ đúng 10 sheet, đặt tên chính xác từng ký tự (ThongTin, YeuCauChucNang, YeuCauPhiChucNang, RangBuoc, MoHinhTrangThai, ThuocTinhAnToan, LoaiTruLanNhau, HamSinhMa, KiemTraSoHoc, TestCase).
- ☐ Mỗi sheet dạng bảng có đúng 1 dòng tiêu đề (dòng 1) với tên cột đúng chính tả như trong Chương 3.
- ☐ Không còn dòng/sheet mẫu (từ file gốc đã copy) chưa được sửa lại theo hệ thống của mình.

5.2. Nội dung từng sheet

- ☐ ThongTin: có đủ name và description mô tả rõ nghiệp vụ + các thành phần đồng thời.
- ☐ YeuCauChucNang: ≥ 5 dòng, phủ đủ 4 DanhMuc (safety/liveness/memory/functional).
- ☐ YeuCauPhiChucNang: ≥ 5 dòng, mỗi dòng có số đo cụ thể.
- ☐ RangBuoc: ≥ 5 dòng; các ràng buộc độ dài dữ liệu khớp với KichThuocBuffer ở HamSinhMa.
- ☐ MoHinhTrangThai: mọi trạng thái đều có ít nhất 1 dịch chuyển đi ra (kể cả tự trỏ về chính nó cho trạng thái kết thúc hợp lệ) — KHÔNG còn deadlock ngoài ý muốn.
- ☐ MoHinhTrangThai: có ít nhất 1 cụm 4 trạng thái free/main_holds/log_holds/conflict với nhãn "violation" ở trạng thái conflict.
- ☐ ThuocTinhAnToan: mỗi cụm khóa có đúng cặp AG(!violation) + EF(<...>_locked); mỗi vòng đời nghiệp vụ "đang xử lý" có 1 công thức AG(...->AF(...)).
- ☐ ThuocTinhAnToan: mọi nhãn dùng trong công thức đều khớp chính xác (không dấu, đúng hoa/thường) với nhãn đã khai báo ở MoHinhTrangThai.
- ☐ LoaiTruLanNhau: dòng đầu tiên là cặp nhãn của cụm khóa CHÍNH; ≥ 5 dòng tổng cộng.
- ☐ HamSinhMa: có đủ cả 4 loại lỗi C (buffer-overflow, memory-leak, double-free, use-after-free) và cả 2 loại Python (divide, magic_value).
- ☐ HamSinhMa: KichThuocBuffer của mỗi hàm buffer_copy khớp với ràng buộc độ dài tương ứng ở sheet RangBuoc.
- ☐ KiemTraSoHoc: ≥ 5 dòng; mỗi dòng có DieuKien dùng đúng công thức <ngưỡng theo bảng mục 3.9> và BieuThucTran là tổng các biến tương ứng.
- ☐ TestCase: ≥ 5 dòng, mọi YeuCauLienQuan trỏ tới ID có thật ở sheet YeuCauChucNang/YeuCauPhiChucNang/RangBuoc; mỗi bước kiểm thử trên 1 dòng riêng trong ô CacBuoc.

5.3. Kiểm tra bằng chạy thử

- ☐ File nạp được không lỗi bằng parse_specification_excel() (không có SpecificationError).
- ☐ run_pipeline() chạy hết cả 4 chương, không có ngoại lệ (exception) không kiểm soát được.
- ☐ Báo cáo cuối cùng (report.to_text()) liệt kê đủ các CWE dự kiến theo Chương 4 của tài liệu này.

- ☐ Mục “[Test Case] Truy vết từ đặc tả” trong báo cáo hiển thị đủ số test case đã khai báo.

Chương 6 — Phụ lục: lệnh kiểm tra nhanh

6.1. Kiểm tra nhanh cấu trúc workbook bằng Python (không cần chạy pipeline)

Đoạn script sau liệt kê tên sheet và số dòng dữ liệu mỗi sheet — dùng để tự rà soát nhanh trước khi nạp vào pipeline:

```
import openpyxl
wb = openpyxl.load_workbook("duong_dan/file_cua_ban.xlsx", data_only=True)
print("Sheets:", wb.sheetnames)
for ten in wb.sheetnames:
    ws = wb[ten]
    so_dong_du_lieu = max(0, ws.max_row - 1)
    print(f"  {ten}: {so_dong_du_lieu} dong du lieu")
```

6.2. Nạp và chạy toàn bộ pipeline, in báo cáo đầy đủ

```
from core.spec_excel import parse_specification_excel
from core.spec_pipeline import run_pipeline

spec = parse_specification_excel("duong_dan/file_cua_ban.xlsx")
print(f"Da doc dac ta '{spec.name}' "
      f"({len(spec.requirements)} yeu cau, {len(spec.test_cases)} test case).")

report = run_pipeline(spec, fuzz_iterations=300, seed=1)
print(report.to_text())
```

Hoặc chạy trực tiếp bằng module (không cần viết script):

```
python3 -m core.spec_excel duong_dan/file_cua_ban.xlsx
```

6.3. Xuất ngược từ JSON sang Excel (tạo bộ khung để điền tiếp)

Nếu đã có sẵn một đặc tả JSON (theo dataset/specs/FORMAT.md) và muốn chuyển sang Excel để tiếp tục điền/chỉnh sửa theo giao diện bảng tính quen thuộc:

```
from core.spec_pipeline import parse_specification_file
from core.spec_excel import export_specification_to_excel

spec = parse_specification_file("dataset/specs/ten_file.json")
export_specification_to_excel(spec, "output/ten_file_da_xuat.xlsx")
```

Phụ lục A — Nguồn tham khảo & phạm vi trích dẫn

Toàn bộ ví dụ minh họa trong tài liệu này (trừ khi ghi chú khác) được trích từ bộ đặc tả mẫu quanly_thuvien_muontra do người dùng cung cấp (quanly_thuvien_spec_full.xlsx) — một hệ thống Quản lý Thư viện xử lý nghiệp vụ mượn/trả sách, đã được kiểm tra có cấu trúc hợp lệ theo đúng 10 sheet mô tả trong Chương 3, với quy mô đầy đủ như sau:

Sheet	Số dòng dữ liệu trong bộ mẫu đầy đủ
ThongTin	2 (name, description)
YeuCauChucNang	30
YeuCauPhiChucNang	30
RangBuoc	30
MoHinhTrangThai	58
ThuocTinhAnToan	30
LoaiTruLanNhau	30
HamSinhMa	30
KiemTraSoHoc	30
TestCase	32

Các tài liệu kỹ thuật gốc được tham chiếu trực tiếp: spec_verification_lab/core/spec_excel.py, spec_verification_lab/core/spec_pipeline.py, spec_verification_lab/core/static_smt.py, spec_verification_lab/core/kripke.py, spec_verification_lab/dataset/specs_excel/FORMAT_EXCEL.md, spec_verification_lab/dataset/specs/FORMAT.md.